Every Agent 项目介绍

> **哪里都可以使用的 Agent** —— 开源、可自托管（self-hosted）的远程 AI Agent 控制平台：让 AI 任务跑在你自己的电脑上，然后用手机、平板、办公室电脑、任何浏览器的标签页随时查看进度、继续对话、回答它的提问。

一、项目背景

1.1 现实痛点

AI Agent（智能体）正在从"聊天机器人"进化为"能替人干活的数字员工"——写代码、改文件、跑测试、提交 git。但当前主流方案各有各的别扭：

- **云端 Agent 平台**：任务跑在厂商服务器上，代码、文件、凭证都要上传到第三方，数据主权无从谈起；且按 token 计费，长任务成本高昂；
- **本机跑的 Coding Agent**（终端里的 CLI 工具）：数据和成本问题解决了，但人必须守在这台电脑前——AI 每遇一次授权、每提一个问题，都得有人在屏幕前点确认。任务跑两小时，人就被钉在椅子上两小时；
- **想远程遥控自己电脑**？需要公网 IP、端口转发、内网穿透——NAT 和路由器把绝大多数家庭用户挡在门外；
- **断线即丢进度**：浏览器关了、Wi-Fi 切了、网络抖了，任务输出缺一段少一截，多端之间更谈不上同步。

1.2 项目定位

Every Agent 的答案是一套「**公网可及、本机执行**」的 AI Agent 系统，一句话：**自己的 Agent，哪里都能用**。

- **开源 · 免费 · 自托管**：代码、文件、数据全部留在用户自己的电脑上，不经过任何第三方服务器，适合重视隐私与数据主权的个人开发者和小团队；
- **远程控制，不需要公网 IP**：NAT、路由器、无公网 IP 都不用操心——worker 只需「打出去」一条 WebSocket 加密长连接，世界各地的浏览器都能遥控它；

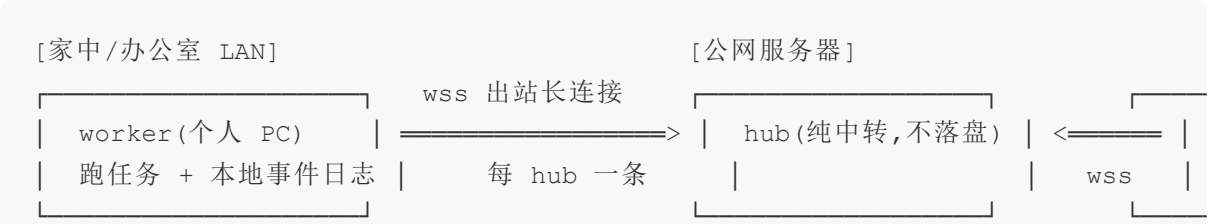
- **多端同步、断线续播**：关掉浏览器任务照跑，重开后从头到当前完整可见、继续流式输出；
- **自带安全护栏**：命令在系统级沙箱中执行、越界操作弹窗授权，可选 AI 审议 / 无人值守自动裁决。

项目采用 Apache License 2.0 开源协议，当前版本已实现完整的插件化架构（v1.0.0），并进入"用 Every Agent 开发 Every Agent"的自举（dogfooding）阶段。

二、技术架构

2.1 总体架构

系统采用「**一台公网中转 + 一台本机执行 + 任意多端浏览器**」的三角拓扑：



worker 主动向公网 hub 发起**出站** WebSocket 加密长连接，前端再连到 hub——hub 在两者之间纯转发消息帧，自身零状态、零缓冲、零业务逻辑、零 ack。

2.2 五个模块，自由组合部署

模块	职责	技术要点
`every-agent-hub`	公网消息中心：纯中转 WebSocket	零状态、零缓冲、零业务逻辑、零 ack
`every-agent-worker`	执行器：托管任务/模型/工作区/沙箱	Spring Boot + Spring AI 2，任务跑在虚拟线程
`every-agent-web`	前端：React + TS，内置客户端 SDK	桌面/移动端响应式适配
`every-agent-contract`	纯协议契约：帧/RPC 信封/错误码	Java + TS 双语言，三层零依赖解耦

模块	职责	技术要点
`every-agent-desktop`	Electron 桌面版：三件套一体打包	Windows x64 安装包 / 便携版，开箱即用

三层解耦是硬约束：hub / worker / client 互相零依赖，只依赖 `every-agent-contract` 纯协议契约；版本统一由根 pom 锁定。部署形态可按需裁剪：单机用 desktop 一体包；公网多端远程则公网服务器只跑 hub + web，自己电脑单独跑 worker。

2.3 关键技术设计

（1）连接层：出站长连接穿透 NAT

worker 无需暴露任何入站端口，只向每个 hub 维持一条出站 wss 长连接（`worker.hubs` 列表支持同时注册多个 hub，互为冗余）。前端到 hub 也是 wss。全程加密，无需公网 IP 与端口转发。

（2）数据层：本地事件日志为唯一真相源

- 任务/对话数据由 worker 落盘到本地 `workspaces/<workspaceId>/tasks/<taskId>/`，hub 不落任何盘——hub 宕机、断网都不影响任务继续跑完；
- 任务流采用**混合模型**：实时增量经 stream 频道定向推送（ext.target=sessionId），历史/补齐走 `task.poll` 拉取，两路同源、按 seq 去重；
- seq 是任务流事件空间：每任务从 1 单调递增、跨运行延续，重连后自动重订阅 + 拉取补齐，实现**断线续播、永不丢进度**；运行中任务日志永不修剪，任务永久保留。

（3）执行层：Spring AI 2 框架 + 虚拟线程

- Agent 执行完全复用 Spring AI 2 的 `ChatClient` + `Advisor` 生态，不手搓 agent 循环；一个 Advisor 只负责一个功能（注入 skill、发射事件、记忆、护栏），能力增强一律以 Advisor 形式插入；
- 任务跑在 Java 25 虚拟线程上，发布/落盘永不阻塞任务线程；
- 子 Agent 与主 Agent 共用同一运行入口与 Advisor 链，仅 agentId 不同，支持进程内嵌套、上下文隔离、并发运行。

（4）安全层：双道鉴权 + 系统级沙箱 + 人机协作护栏

- **两道独立密钥，各管一段**：hub key 管「连上 hub」，worker apiKey 管「动这台 worker 的数据」——泄露一把不牵累全局；频道即鉴权边界，`u.<ownerKey>.` 前缀 ACL，文件/git/任务操作一律 jailed 到工作区根；

- **系统级沙箱**：不是应用层路径检查，而是 OS 底层隔离。默认 `sandbox-windows-codex` 后端仿 Codex 双账户 + WRITE_RESTRICTED 受限令牌 + Capability SID，工作区之外物理上只读不可写；可选 `wsl-ubuntu`（宿主盘完全不可见、可整体重装、唯一能真正断网的后端）与 `windows-mic`（Medium IL 轻量降权），`worker.sandbox.type` 一行切换；
- **人机协作护栏**（沙箱之上）：工作区外路径访问/危险命令动词一律经 PermissionGate 弹窗授权（拒绝/本轮运行/本任务三档），可开 AI 审议自动裁决、无人值守全自动放行；git 凭证 AES-GCM 加密存本机。沙箱保证 AI 弄不坏系统，护栏保证 AI 不偷偷干用户不想让它干的事。

（5）插件层：一切能力皆可插拔

沙箱、git、模型池、子 Agent、上下文压缩、AI 审议、无人值守、任务队列等"核心能力"没有一样写死在宿主里，全部以插件形式实现，仓内自带 26 个内置插件：

- 后端 15 个扩展点：工具 / 沙箱后端 / Advisor / 模型增强 / 任务生命周期 / 自定义 RPC / 斜杠命令；
- 前端 12 个 UI 扩展点：侧边栏面板 / 工作区标签 / 工具调用视图 / 文件编辑器 / 消息动作等；
- 三种插件形态（纯 Java / 纯 Web / 混合），`create-everyagent-plugin` 脚手架 5 分钟起步，打包成 `.eap` 经前端「扩展」面板一键安装，经 URLClassLoader 类加载隔离。

2.4 技术栈

层	技术
hub	Java 25 · Spring Boot WebFlux（Reactor Netty WS）
worker	Java 25 · Spring Boot · Spring AI 2（ChatClient + Advisor） · 虚拟线程 · JDK HttpClient WS
web	React · TypeScript（内置 TS 客户端 SDK）
desktop	Electron · electron-builder（NSIS + portable）
模型	任意 OpenAI 兼容 provider：OpenAI、DeepSeek、Qwen、GLM、本地 vLLM/Ollama…

三、应用场景

3.1 个人开发者：多端远程开发（核心场景）

> "我让 AI 干活，但我不想守在电脑前。"——这是 Every Agent 的立身之本。

晚上在书房 PC 上扔给 Every Agent 一个"重构这个模块并跑通测试"的任务，然后去客厅沙发上用手机看进度。中途 AI 遇到一个工作区外的写入，弹授权窗——在手机上点"本任务放行"即可，不用走回书房。第二天通勤路上用平板查看最终 diff，顺手 merge。AI 需要确认时会在所有在线前端同步弹出问题，哪个端都能作答。

3.2 重视隐私的个人与小团队：数据不出本机

hub 只是纯中转，不落盘、不理解内容——它连"任务"是什么都不知道。任务日志、文件、git 凭证、模型配置全部存在 worker 本机。适合律所、医疗、财务等数据合规敏感场景，以及单纯不想把代码上传到任何第三方的开发者。

3.3 企业内部私有化部署：接入已有系统零负担

一台服务器即可完成私有化：公司内部部署 hub + worker，员工打开浏览器就能用，数据全程留在企业内网。对上游系统而言，平台就是 WebSocket + RPC（协议契约完全开放），OA、办公门户、小程序、企业微信应用皆可集成。平台认证故意只做 apikey、不做用户账号模型——企业接入已有 SSO/OA 用户体系时，只需在接入层把员工身份映射为 apikey，集成成本极低。

3.4 高可靠与多环境：多 worker / 多 hub 组合

一个前端可统一管理 hub 下所有在线 worker（家里一台 + 办公室一台）；一台 worker 可同时注册公网 hub + 内网/备用 hub，同一份任务与文件多 hub 同步可见、互为冗余；desktop 内置的 worker 同样支持多 hub——"开箱即用的单机"与"到哪都能遥控的远程"同步成立。

3.5 零成本开发：免费模型池容灾

任意 OpenAI 兼容 provider 改一行配置即可接入；模型池让一个任务配多个模型，主模型失败自动切换下一个，任务不中断。项目后期就是靠免费模型（商汤「日日新」SenseNova 免费额度、OpenRouter 免费端点）开发出来的——额度用完自动切换，开发不中断。

3.6 其他日常能力

任务永久保留、运行即销毁、冷启动续跑；输入队列管理、斜杠命令与任务级开关；工作区文件树、git 全流程操作；多工作区并行管理多个项目；任务完成/错误/提问/授权请求的多端通知。

四、创新点

（1）出站 WebSocket 反向连接：免公网 IP 的远程遥控

不同于需要公网 IP/端口转发/内网穿透工具的传统远程方案，worker 只需一条出站长连接即可被全球任意设备遥控——把"内网设备"变成"公网可及的 Agent 节点"，且传输全程 wss 加密。

（2）hub 极简主义 + worker 本地事件日志：真正的断线续播

hub 零状态、零缓冲、零业务逻辑、零 ack，一切补齐依赖 worker 本地事件日志 + `task.poll` 拉取；seq 单调递增事件空间 + 实时推送/历史拉取混合模型，保证关浏览器、切网、hub 重启后任务流**从头到当前完整可见**且继续流式输出。数据主权（本机落盘）与多端可达（公网中转）这对矛盾被同时满足。

（3）系统级"真沙箱"：OS 底层隔离，而非应用层模拟

	普通 coding agent（无沙箱/软隔离）	Every Agent（系统级沙箱）
工作区外写入	靠路径检查，一条 `cd` 就绕过	OS 层禁止，进程压根没有越界写权限
进程逃逸	无防护	受限令牌/独立发行版，攻击面大幅收窄
系统文件/用户目录	AI 可误删	只读不可写，误操作物理上不可能
恢复性	垃圾残留要手动清	WSL 发行版可一键重装，宿主盘完全不可见

思路参考 OpenAI Codex 的双账户 + 受限令牌方案，以纯 Windows 原生机制实现（无需 WSL/虚拟机，启动快、开销低）；即使用户主动关闭沙箱，PermissionGate 责任链仍然兜底拦截越界写。

（4）双道独立鉴权体系

hub key 与 worker apiKey 两把钥匙各管一段、按 worker 隔离，泄露一把不牵累全局；频道即鉴权边界 + 工作区 jail，把攻击面切分成互不感染的独立区间。

（5）"一切能力皆插件"的开放架构

沙箱、模型池、子 Agent 这些在别处是"硬内核"的能力，在本项目中全部是跑在统一插件体系上的普通插件（26 个内置插件即 26 份活教材）；后端 15 + 前端 12 个扩展点允许不改一行宿主代码地为整个系统加能力，`.eap` 一键安装 + 类加载隔离兼顾了易用与安全。

（6）完全复用 Spring AI 框架的工程红线

不自研 agent 循环、工具调用循环、响应聚合，一切经由 `ChatClient` + `Advisor` 生态表达；执行核心保持"很薄的一层"，能力增强一律做成单一职责的 Advisor——这在 AI 应用工程化上是少见的克制设计，显著降低了维护成本与框架漂移风险。

（7）自举（Dogfooding）+ 免费模型友好

项目后期完全使用 Every Agent 自己开发 Every Agent；配合模型池容灾，"白嫖免费模型跑长任务"成为可持续的开发方式——证明了本平台作为生产力工具的完备性。

五、总结

Every Agent 用「公网中转 + 本机执行」的拓扑，同时拿到了三个过去不可兼得的东西：**数据主权**（一切数据留在本机）、**多端可达**（任何浏览器随时遥控）、**进度可靠**（断线续播、永不丢失）。系统级沙箱与人机协作护栏让它敢于放手干活，插件化架构让它可以无限生长——对企业是可私有化集成的 Agent 基础设施，对个人是"扔下任务就走、躺在哪都能盯"的数字员工遥控器。

> 项目采用 Apache License 2.0 开源。详细信息参阅仓库 `README.md` 与架构文档 `docs/ARCHITECTURE.md`（唯一架构事实源）。